IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 30 September 2025, accepted 19 November 2025, date of publication 2 December 2025,
date of current version 22 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3638251

RESEARCH ARTICLE

ProveRAG: Provenance-Driven Vulnerability Analysis With Automated Retrieval-Augmented LLMs

REZA FAYYAZI[1], (Graduate Student Member, IEEE), STELLA HOYOS TRUEBA[2], MICHAEL ZUZAK[1], (Member, IEEE), AND SHANCHIEH JAY YANG[3], (Senior Member, IEEE)

[1]Department of Computer Engineering, Rochester Institute of Technology, Rochester, NY 14623, USA
[2]Department of Computer Science, Karlsruhe Institute of Technology, 76131 Karlsruhe, Germany
[3]Informatics and Applied Technology Institute, Gonzaga University, Spokane, WA 99258, USA

Corresponding author: Reza Fayyazi (rf1679@rit.edu)

This work was supported by the National Science Foundation under Grant 2344237, Grant 2228001, and Grant DGE-2125362.

ABSTRACT In cybersecurity, security analysts constantly face the challenge of mitigating newly discovered vulnerabilities in real-time, with over 300,000 vulnerabilities identified since 1999. The sheer volume of known vulnerabilities complicates the detection of patterns for unknown threats. While LLMs can assist, they often hallucinate and lack alignment with recent threats. Over 40,000 vulnerabilities have been identified in 2024 alone, which are introduced after most popular LLMs' (e.g., GPT-5) training data cutoff. This raises a major challenge of leveraging LLMs in cybersecurity, where accuracy and up-to-date information are paramount. Therefore, we aim to improve the adaptation of LLMs in vulnerability analysis by mimicking how an analyst performs such tasks. We propose **ProveRAG**, an LLM-powered system designed to assist in rapidly analyzing vulnerabilities with automated retrieval augmentation of web data while self-evaluating its responses with verifiable evidence. ProveRAG incorporates a self-critique mechanism to help alleviate the omission and hallucination common in the output of LLMs applied in cybersecurity applications. The system cross-references data from verifiable sources (NVD and CWE), giving analysts confidence in the actionable insights provided. Our results indicate that ProveRAG excels in delivering verifiable evidence to the user with over 99% and 97% accuracy in exploitation and mitigation strategies, respectively. ProveRAG guides analysts to secure their systems more effectively by overcoming temporal and context-window limitations while also documenting the process for future audits.

INDEX TERMS ProveRAG, LLM, provenance, CVE, CWE, RAG, vulnerability, self-critique.

I. INTRODUCTION

In today's fast-paced cybersecurity landscape, security analysts are continuously challenged to understand and mitigate newly discovered threats in real-time [1]. Common Vulnerabilities and Exposures (CVEs) [2] serve as a critical framework for identifying and detecting known vulnerabilities. With over 300,000 CVEs cataloged since 1999, the sheer volume makes it increasingly difficult and time-consuming for analysts to detect patterns and respond effectively to emerging threats. Public vulnerability databases, such as the NIST National Vulnerability Database (NVD) [3], predominantly rely on labor-intensive manual processes to analyze and categorize vulnerabilities. This reliance on human effort can cause delays in threat identification and response, which makes it difficult to keep pace with the rapidly evolving threat landscape [1].

For example, the *CVE-2024-0302* is a critical vulnerability that, if left unchecked, could lead to deserialization of untrusted data. To mitigate this vulnerability, fields should be made transient to be protected from deserialization. However, protecting an enterprise network necessitates discovering which vulnerabilities are relevant by combing through over 300,000 threats aggregated in these databases, with more

The associate editor coordinating the review of this manuscript and approving it for publication was Tyson Brooks.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

added each day. Separate mitigation databases, such as the Common Weakness Enumeration (CWE) [4], must also be explored to identify, implement, and deploy mitigations to the system. Due to the sheer volume of information, manually identifying and mitigating threats is challenging, and automating the identification and mitigation process is essential to reduce risk and improve overall resilience.

Large Language Models (LLMs) have been widely adopted in cybersecurity applications [5], [6], [7], [8]. They have been particularly effective in assisting with the analysis of Cyber Threat Intelligence [5], [9], supporting LLM-assisted attacks [8], [10], [11], and improving vulnerability detection [6], [7].

However, prior work has identified three aspects that limit the broader adoption of LLMs to cybersecurity applications [12], [13], [14], [15], which we aim to address in this work. These limitations are: 1) *Hallucination:* the tendency to generate misleading information that appears plausible but is factually incorrect. Some works have shown the consequences of this problem [12], [13]. 2) *Temporal Knowledge:* the failure of LLMs to respond accurately when they are exposed to data that come after the cutoff point of their training data [14], [16]. In cybersecurity, daily emerging vulnerabilities—over 40,000 identified in 2024 alone—pose serious risks, especially if they appear after LLMs' training data cutoff. Retrieval Augmented Generation (RAG) techniques have been proposed to address this problem [17] by retrieving relevant external information. However, adoption of RAG often leads to 3) *Limited Context Windows:* which restrict the number of tokens that can be provided to prompt the model, thus limiting RAG-based temporal alignment approaches [15]. These limitations constitute a major challenge of leveraging LLMs in cybersecurity, where accuracy, time, and up-to-date information is essential.

In this work, we aim to improve the adaptation of LLMs in vulnerability analysis. To do so, we propose **ProveRAG**, an advanced LLM-powered system designed to assist security teams in rapidly analyzing CVEs with verifiable evidence. What sets ProveRAG apart is its ability to cross-reference, rationalize, and validate information from authoritative sources, such as NVD and CWE. By doing so, we demonstrate that ProveRAG exhibits fewer instances of omission and hallucination through self-critiquing its own response.

A key innovation in ProveRAG is its incorporation of **provenance** into its analysis. By documenting the source of every piece of information, ProveRAG **reveals** the hallucinations and inaccuracies common in LLMs' responses and gives security analysts the confidence to make informed decisions. For example, when suggesting mitigation strategies for vulnerabilities like *CVE-2024-0302*, ProveRAG reveals pieces of data from trusted databases (such as NVD and CWE) that led to its generated response. This provides a clear audit trail that can be reviewed and verified, thus fostering trust and enabling effective action.

We test ProveRAG on 2024 CVEs, and we compare with two different baseline approaches: 1) prompt-only and 2) chunking retrieval during generation We compare these baselines to our proposed summarizing technique (integrated into ProveRAG). The prompt-only technique is directly asking the LLM details about a specific CVE exploitation and mitigation strategies with data after its training cutoff knowledge. For the chunking technique, we enhance information retrieval by splitting and embedding the retrieved content into smaller chunks. However, this approach requires a very large context window and can include redundant information that might mislead the LLM.

Therefore, we design a summarizing technique to process CVEs, inspired by [**?**]db@bib:18 and bib:19 in an automatic way by mimicking how security analysts perform vulnerability analysis. To do so, we start by exploring the NVD website and summarize the content with respect to a specific CVE-ID. Next, we move to the associated CWE link, and further to the relevant references (hyperlinks) to provide summaries with respect to the exploitation and mitigation questions (emulating the typical workflow of a security analyst). A key advantage of the summarizing technique is its efficiency in handling large-scale data without relying extensively on document chunking, which enhances both retrieval quality and the accuracy of responses (while handling the context window limitation). This makes the system particularly suited to better guide analysts in a timely manner while addressing more complex cybersecurity queries - such as those related to CVE mitigation that require integrating more external information.

ProveRAG has the ability to provide verifiable evidence (TPs), while also **revealing** hallucinations (FPs) and failed retrievals (FNs) to determine whether a generated response is accurate and trustworthy (the provenance part of the proposed framework). Furthermore, we demonstrate that by providing the LLM with highly-structured context (Aqua Vulnerability Database [20]), ProveRAG's performance increases significantly. Our testing shows that ProveRAG achieves outstanding detection accuracy, with 99% for exploitation and 97% for mitigation strategies.

ProveRAG not only enables more reliable decision-making but also documents its processes to ensure a robust audit trail for future reviews. For evaluation, we ensure comprehensive coverage of all relevant information for each CVE by incorporating external resources beyond the NVD and CWE. Finally, we show that the proposed ProveRAG framework generalizes across different models by comparing *OpenAI's gpt-4o-mini* and *Meta's Llama-3.1-8B* [21]. This allows security analysts to rapidly navigate the massive amounts of data to identify relevant cybersecurity vulnerabilities applicable to their enterprise system and develop appropriate mitigation strategies. The contributions of our work are as follows:

- We propose ProveRAG, an LLM-powered system developed to self-critique its generated responses with rationale, by providing pieces of evidence from verifiable sources that led to its generation (i.e., provenance).
- We apply ProveRAG to perform vulnerability analysis, a critical application that requires retrieval and verification of new, unseen knowledge.

- We demonstrate that ProveRAG not only can **show proof** for its responses (TPs), but also **reveals** hallucinations (FPs) and failed retrievals (FNs) with evidence.
- We show that ProveRAG effectively respond to unseen data while relaxing the limitations imposed by context windows, leading to high #TPs (99% accuracy) and low #FPs and #FNs, even when answering complex queries (e.g., vulnerability mitigation).
- We evaluate ProveRAG on two LLMs, *gpt-4o-mini* and *Llama-3.1-8B*, to support the generalizability of the proposed framework.

II. BACKGROUND & RELATED WORK

A. LARGE LANGUAGE MODELS

The significant advancements in Large Language Models is largely attributed to the integration of the Transformer architecture [22]. These models are characterized by their extensive training on vast and diverse datasets, leading them to generate coherent and contextually relevant text that closely mirrors human language. OpenAI's GPT-4o [23] and Meta's LLama-3 [24] are leading examples in this category. The ability of these models to manage diverse and complex tasks has been extensively documented in the literature [25], [26]. However, despite the efficacy of LLMs in diverse tasks, these models have shown to be prone to hallucination in their responses [12], [13].

B. RETRIEVAL AUGMENTED GENERATION

The hallucination issue of LLMs stems from their reliance on pre-trained knowledge, which causes another limitation and that is the temporal scope of their training data. Such limitations pose significant challenges in fields requiring high precision, such as cybersecurity. To address these challenges and provide more factual knowledge, RAG was introduced [17]. RAG effectively combines search algorithms with LLM prompting: the model is prompted to answer a query using relevant information retrieved from external resources as context. This enhances LLMs' ability to generate accurate and contextually relevant responses.

Several advanced RAG techniques have been proposed to further refine this integration [18], [19], [27], [28], [29], [30]. FLARE [27] improves model accuracy by predicting the next sentence and using low-confidence tokens as queries to re-retrieve relevant documents. DRAGIN [28] leverages the LLM's real-time information needs with self-attention to decide what and when to trigger retrieval. Adaptive-RAG [29] employs a smaller LLM as a classifier to assess query complexity and subsequently selects the most appropriate retrieval strategy. Self-RAG [30] employs self-reflection for retrieved documents to improve generation quality. However, Self-RAG trains two models–a critic model and a generator model– to assess its predictions. Our approach avoids any fine-tuning of LLMs, ensuring an automatic process designed to match the rapid pace in cybersecurity applications. Summarizing Retrievals (SuRe) [18] constructs summaries of retrieved passages for multiple answer candidates and confirms the most plausible answer by evaluating and ranking the generated summaries. TC-RAG [19] integrates a memory stack system with backtracking and summarization functions to correct errors and eliminate redundant information. Despite these advancements, all these methods face the challenge of effectively showcasing the evidence underlying their generated responses.

C. LLMs IN VULNERABILITY ANALYSIS

There are multiple works that integrated the use of LLMs in vulnerability analysis [7], [31], [32], [33]. Cheshkov et al. [7] used GPT models to identify CWE vulnerabilities in Java code and revealed that LLMs struggle with detecting vulnerable code. In contrast, Khare et al. [31] demonstrated with multiple prompt strategies that LLMs can outperform deep learning models in vulnerability detection when carefully prompted, which indicates the importance of prompt engineering. Building on this, Lu et al. [33] introduced GRACE, a framework using in-context learning and graph-structured information with LLMs, which showed improvements in vulnerability detection on C/C++ datasets. Du et al. [32] introduced Vul-RAG, LLM-based vulnerability detection technique that constructs a comprehensive knowledge base by extracting multi-dimensional insights from existing CVE instances. However, the reliance on human analysis in their approach makes it a manual time-consuming work and limits its efficiency in real-time vulnerability detection. In addition, PentestGPT [8] has shown capabilities in assisting in penetration testing. While it does not address the same problem this paper focuses on, the authors noted that LLMs are constrained by limited token sizes (i.e., context window), preventing long-term memory retention. This work builds on this observation and develop efficient RAG to bring LLM's attention to relevant contents.

III. MOTIVATION & PROBLEM FORMULATION

Vulnerability analysis involves evaluating a system's configuration to identify potential security risks. By assessing attack surfaces and risk factors, it helps uncover weak points in the system that require effective mitigation to prevent exploitation. Traditional approaches to vulnerability analysis typically involve manual assessment and the use of static databases, which can be both time-consuming and prone to human error. Moreover, while existing systems can assess vulnerabilities, they often fall short in providing detailed, actionable insights necessary for effective mitigation. This gap in the process leaves security teams with a critical need for more advanced systems capable of assessing threats, generating comprehensive mitigation scenarios, and documenting evidences of their recommendations. The motivation behind this work stems from these challenges. We seek to address these limitations by answering the following research questions:

RQ1: How can the knowledge cutoff of LLMs be addressed in the context of continuously emerging

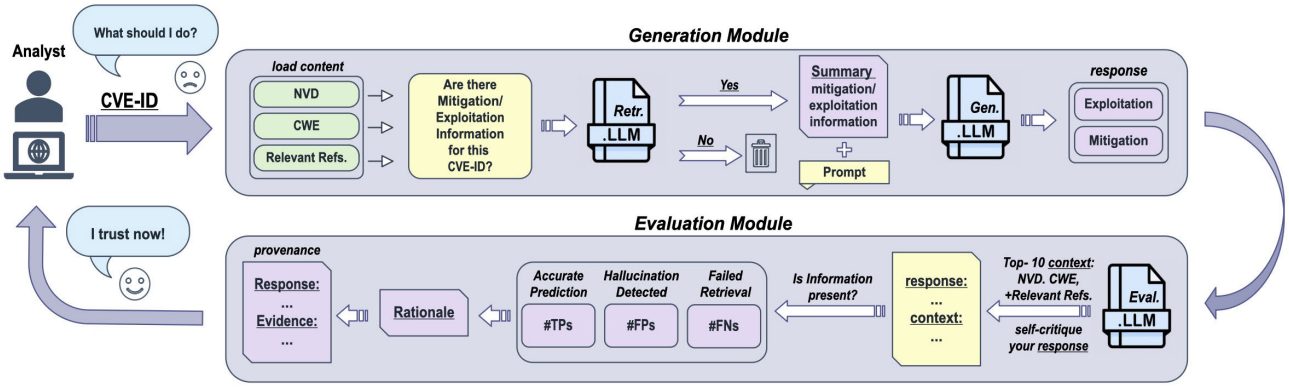


FIGURE 1. ProveRAG: automated retrieval-augmented generation & self-critique provenance.

cybersecurity threats? One of the key problems we address is the knowledge cutoff (temporal limitation) of LLMs. With the constant emergence of new vulnerabilities, it is crucial that security systems remain effective even when dealing with data that falls outside the LLM's training window. Our system, ProveRAG, is designed to overcome this challenge by integrating up-to-date information from verified resources to effectively assess vulnerabilities discovered post-training. It does this by systematically cross-referencing to standardized security databases (e.g., NIST NVD) in a sequential manner and gathering relevant information, particularly for complex queries that require more in-depth and thorough retrieval, such as those related to mitigation strategies.

RQ2: How to manage information overload in the context of RAG-based LLMs for complex queries? Another significant problem we aim to solve is the issue of information overload (i.e., arising from context window limitations). This problem of inability of LLMs to process massive amounts of information and identify relevant information can lead to less effective responses, as discussed in literature [15]. While more information is often seen as beneficial, our research shows that in the context of LLM-powered vulnerability analysis, an abundance of data can lead to inefficiencies (when chunking technique is used). The summarization technique we encountered in ProveRAG, can alleviate this issue. This technique can make the system especially effective in handling more complex cybersecurity issues, such as CVE mitigation queries that require more attention on additional relevant resources.

RQ3: How to enhance the accuracy and trustworthiness of threat analysis while addressing issues related to hallucination and omission errors? One other key motivation for developing ProveRAG is the critical requirement to enhance the accuracy and trustworthiness of vulnerability analysis by mitigating issues related to hallucination and omission errors. ProveRAG addresses this by integrating authoritative sources such as NVD and CWE into its framework. This integration ensures that the recommendations provided by ProveRAG are grounded in reliable and verifiable data, which we call provenance. To leverage the capabilities of LLMs, ProveRAG utilizes an explicit and structured prompt designed with the chain-of-thought technique [34]. Based on our evaluation of ProveRAG (Sec. V-B2), this causes the LLM to substantiate all responses with direct evidence, enhancing the credibility of the output.

IV. PROVERAG METHODOLOGY

The proposed ProveRAG methodology consists of two stages: The Generation and the Evaluation. An overview of each stage is in Figure 1. Note that the *Retr. LLM, Gen. LLM,* and *Eval. LLM* are the same model. For the remainder of this section, we discuss each stage in turn. First, in the generation module, the *Retr. LLM* will load web content from external sources about a CVE vulnerability—starting with NVD, followed by CWE, and then additional references (replication of a typical security analyst workflow in practice) —and summarizes the content with respect to exploitation and mitigation information. Next, the *Gen. LLM* will look at the summaries and generates detailed exploitation and mitigation strategies for that specific CVE. Furthermore, in the evaluation module, the *Eval. LLM* will critique its own response by looking at the content from the official and external (NVD, CWE, and hyperlinks in NVD) websites to show evidence on whether its response were accurate, hallucinated, or omitted critical information. Finally, the LLM will provide provenance by showing similar pieces of information from both the response and the retrieved context that led to its generation, followed by a rationale to ensure reliability and enhance decision-making. It is worth noting that we used the OpenAI's *gpt-4o-mini* [23] model as the LLM for ProveRAG. This model's training data knowledge is up to Oct. 2023, with 128,000 tokens for context window [23]. However, to show the proposed ProveRAG framework can be generalized across other LLMs, we compare the performance of *gpt-4o-mini* with *Llama-3.1-8B* [21] in Sec. V-E. The *Llama-3.1-8B* model is selected as it has the same context window of 128,000 tokens, and a knowledge cutoff date of Dec. 2023 (i.e., no 2024 CVEs are seen in training). Code and data are available at: https://github.com/RezzFayyazi/ProveRAG.

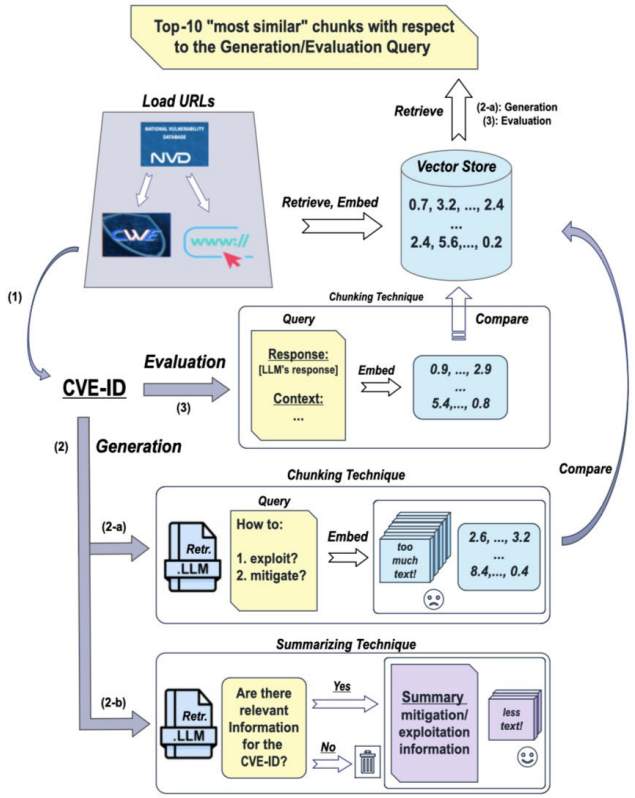


FIGURE 2. **Chunking vs. Summarizing for retrieval process used in both generation and evaluation of ProveRAG.**

A. SYSTEM DESIGN

1) GENERATION MODULE

This module is to query the *Gen. LLM* to provide exploitation and mitigation responses for a given CVE (refer to Figure 1). The following is the generation prompt:

Generation Prompt

You are a cybersecurity expert. Consider the Relevant Information provided below and answer the Query.

Relevant Information: [content of NVD/CWE/Relevant Refs.]

Query: [CVE-xxx-xxx]

Given the specified CVE-ID, please provide detailed answers to the following questions:
1. How can an attacker exploit this vulnerability? Provide a step-by-step description.
2. What are the recommended mitigation strategies for this vulnerability?

The *Relevant Information* is gathered from the summarizing retrieval technique (to answer **RQ1**), and we will show in Sec. V-B1, that it is the better retrieval technique in vulnerability analysis. The *Gen. LLM* will look at the retrieved information for a particular CVE and will provide the response. In the prompt-only experiment, the *Relevant Information* part is removed as we directly query the LLM about a specific CVE. Note that we put the temperature hyperparameter (which assigns the degree of randomness in LLM's output) to zero as we aim to get the most deterministic responses from the LLM.

Chunking vs. Summarizing Retrieval Techniques:

We test on two retrieval techniques (to address **RQ2**). Figure 2 shows how each of these techniques is used. For the chunking technique, the LLM will use the top 10 most similar chunks of 15,000 characters from the resources to the query. In this technique, we split the content of all the URLs into smaller chunks and embed these chunks as vectors, and these embeddings are then indexed to facilitate efficient retrieval. Second, for the summarizing technique, the *Retr. LLM* goes through the NVD, then to the CWE and hyperlinks to summarize the content for the exploitation and mitigation questions The following shows the prompt used in *R*etr. LLM to assess each URL's relevancy, and, if relevant, the LLM will summarize the URL content specifically to address either the Exploitation or the Mitigation question.

Relevancy Prompts

Exploitation Prompt

You are a cybersecurity expert. Your task is to analyze the provided URL content for the [CVE-xxx-xxx] and provide a detailed summary.
Content: content
Please follow the steps below:
Step 1: Assess Relevancy
- Does the content provide relevant information to describe how this [CVE-xxx-xxx] can be exploited?
- Answer: [Yes/No]
Step 2: Summarize Relevant Information
- If the answer is "Yes" in Step 1:
- Summarize the content with step-by-step description to exploit this vulnerability.
- If the answer is "No" in Step 1:
- Summary: NONE

Mitigation Prompt

You are a cybersecurity expert. Your task is to analyze the provided URL content for the [CVE-xxx-xxx] and provide a detailed summary.
Content: content
Please follow the steps below:
Step 1: Assess Relevancy
- Does the content provide relevant information to describe the potential mitigation strategies for this vulnerability?
- Answer: [Yes/No]
Step 2: Summarize Relevant Information
- If the answer is "Yes" in Step 1:
- Summarize the content to describe the potential mitigation strategies for this CVE.
- If the answer is "No" in Step 1:
- Summary: NONE

We further show the performance for different models in Sec. V-E. During runtime, the user's query is first vectorized, and a similarity search is conducted against the indexed chunks. The top-10 results are retrieved and fed as context into the *Gen. LLM's* prompt, which allows the model to answer the user's query with the most relevant content. However, this approach requires a very large context window and can include redundant information that might mislead the LLM.

Therefore, we design an automatic summarization technique, inspired by [?]db@bib:18 and bib:19, for a specific CVE (which is integrated into ProveRAG's architecture). We start by exploring the NVD website and summarize the content with respect to a specific CVE-ID, then move

to the associated CWE link, and finally to the hyperlinks (available in NVD) to provide a summary with respect to the exploitation and mitigation questions. A key advantage of the summarizing technique is its efficiency in handling large-scale data without relying extensively on document chunking, which enhances retrieval quality and accuracy of responses. This makes the system particularly suited for addressing more complex cybersecurity queries, such as those related to CVE mitigation that require exploration of more external resources.

2) EVALUATION MODULE

This module is for the LLM to self-critique its own response (to answer **RQ3**). We carefully and iteratively designed structured and succinct prompts with the chain-of-thought technique [34] to guide the model to think before answering while providing evidence. The following is the evaluation prompt:

Evaluation Prompt

For the [exploitation/mitigation] information of [CVE-xxx-xxx], please evaluate the response based on the provided evidence and select the appropriate value for the defined attributes:

———————-

response: [LLM response]

———————-

evidence: [content from NVD + CWE + Relevant Refs.)]

———————-

As can be seen, the LLM is tasked with providing responses for three attributes, and those attributes are: *value*, *rationale*, and *provenance*. The following are the designed prompts for these attributes:

Evaluation Attributes

Value

- Description: This attribute represents the accuracy of the response based on the provided context about the CVE-ID.
- Allowed Values:
- 'TP' (True Positive): The response fully and accurately reflects the information in the context.
- 'FP' (False Positive): The response contains inaccurate or incorrect information that are not supported by the context.
- 'FN' (False Negative): The response omits information that is present in the context.
- Requirements:
- Must strictly be one of the allowed values: 'TP', 'FP', or 'FN'.
- The selection must be based on an objective comparison between the response and the context.
- Must follow the guidelines for Provenance.

Rationale

- Description: This attribute provides the reasoning behind the selected "value" for the evaluation of the response.

Provenance

- Detailed Instructions for Provenance based on the selected "value":

- If "value" is 'TP':
1. Carefully compare the response with the context.
2. Identify and extract the key segments from both the response and the context that align perfectly.
3. Format the provenance as follows:
- 'response: "[Extracted segment from the response that matches the context]".'
- 'context: "[Corresponding segment(s) from the context]".'

- If "value" is 'FP':
1. Review the response thoroughly to identify parts that are incorrect or not present in the context.
2. Format the provenance as follows:
- 'response: "[Incorrect or unsupported segment from the response]".'
- 'context: "[The closest matching part from the context, or indicate 'No corresponding information in context' if none exists]".'

- If "value" is 'FN':
1. Examine the context to find essential information that is present but omitted in the response.
2. Format the provenance as follows:
- 'response: "[The entire response]".'
- 'context: "[Relevant segment from the context that should have been matched]".'

Guidelines:

- In all cases, ensure the provenance provided is clear, concise, and directly supports the selected "value".
- The provenance should be an exact match or a direct comparison between the response and the context, following the formatting rules strictly.

For these attributes, we ask the LLM to provide a *value*: True Positive (TP) - indicating accurate prediction, False Positive (FP) - indicating the detection of hallucination, or False Negative (FN) - indicating failed retrieval given the response and the evidence (from NVD+CWE+hyperlinks). Next, the LLM is asked to provide a *rationale* for the selected *value*, and finally the *provenance* attribute where the LLM is tasked to show evidence for its response. Note that for provenance, the document chunking technique is utilized as we need the actual information from reputable sources (not the LLM-generated summaries). o

In addition, we ensure that the LLM for ProveRAG do not miss any essential information when finding evidence by utilizing almost the entire context window (128,000 tokens) for provenance. Therefore, for each exploitation and mitigation response separately, we retrieve the top-10 chunks from the NVD, CWE, and hyperlinks with respect to the CVE-ID as evidence, and we compare them with the response using the *Eval. LLM*. All prompts are submitted in a single batch, without any retention of memory between queries.

V. EVALUATION OF PROVERAG

To answer each research question, we discuss the curation of the dataset for (**RQ1**) and provide results for different RAG techniques (**RQ1 and RQ2**). Furthermore, we support the efficacy of provenance by using evaluation metrics (**RQ3**), and we evaluate ProveRAG on another LLM to assess generalizability. Finally, we will show specific examples to demonstrate ProveRAG's effectiveness.

A. DATASET & EXPERIMENT DESIGN

To address **RQ1**, we curated a dataset of critical CVEs reported in 2024 (up until July 25). We focused on CVEs with a Common Vulnerability Scoring System (CVSS) score of 9.0 or higher, which indicates *critical* severity. From this dataset, we further refined our selection by including only those CVEs that had a direct mapping to the CWE database (as CWE contains information about potential mitigation strategies).

TABLE 1. Comparison between Summarizing and Chunking techniques for retrieving the content for CVEs (Provenance Features: TP: accurate prediction, FP: detected hallucination, FN: failed retrieval).

	Evidence Source (NVD+CWE)						
2024 CVEs	**Exploitation**			**Mitigation**			
Methodology	**#TPs**	**#FPs**	**#FNs**	**#TPs**	**#FPs**	**#FNs**	**#Data**
Prompt-only (no retrieval)	51 (11%)	400	31	29 (6%)	335	118	482
Chunking: NVD Only	481 (99%)	0	1	139 (29%)	49	294	482
Chunking: NVD+CWE	476 (99%)	5	1	262 (54%)	58	162	482
Chunking: NVD+CWE+Refs	472 (98%)	10	0	227 (47%)	54	201	482
Summarizing: NVD+CWE+Refs	**476 (99%)**	**6**	**0**	**379 (79%)**	**68**	**35**	**482**

Note that CWE is addressing mitigation at the weakness level, not the vulnerability level.

Moreover, we leveraged the hyperlinks available on the NVD website for each CVE, which provide additional relevant information for each CVE. These hyperlinks were gathered to further enrich the dataset used in the ProveRAG framework, to ensure that we are cross-referencing all available sources of information related to each CVE.

In total, the curated dataset comprised of **482 CVEs**, each mapped to its corresponding CWEs and supplemented with relevant NVD resources. This dataset forms the basis for our temporal alignment and provenance analysis for ProveRAG since the *gpt-4o-mini* and *llama-3.1-8B* models have a knowledge cutoff in September and December 2023, respectively. This allows for a thorough assessment of the LLM's ability to generate accurate, up-to-date, and contextually relevant security recommendations.

B. EFFECTIVENESS OF PROVERAG

1) CHUNKING VS. SUMMARIZING RETRIEVALS

To answer **RQ2**, for the evaluation of retrieval techniques, we compare multiple chunking techniques by increasingly augmenting the *Retr. LLM* with more external resources versus the summarizing technique (including all relevant resources) to assess their abilities to provide accurate exploitation and mitigation strategies. To make a fair comparison between the chunking vs. summarizing technique, we only used NVD and CWE as evidence sources of information. Table 1 shows the overall results for these two techniques.

The prompt-only baseline exhibited poor performance for both exploitation and mitigation, demonstrating the limitations of LLMs with unseen vulnerabilities (as highlighted in RQ1). By integrating the retrieval techniques, we can see that the temporal alignment of the LLM is improved, indicating that such an approach alleviates temporal limitations for cybersecurity applications. For exploitation, all RAG configurations achieved high accuracy, though performance slightly declined when additional data were introduced with the chunking approach. Mitigation queries, however, revealed greater variation: the NVD-only setting achieved 29% accuracy, consistent with its limited mitigation content. Augmenting with CWE substantially increased #TPs (139→262), while adding further URLs reduced performance (7%) due to lower segment quality.

TABLE 2. ProveRAG results for the 482 CVEs.

	Evidence Source (NVD + CWE + Relevant Refs.)					
2024 CVEs	**Exploitation**			**Mitigation**		
Methodology	**#TPs**	**#FPs**	**#FNs**	**#TPs**	**#FPs**	**#FNs**
ProveRAG	460 (95%)	22	0	384 (80%)	66	32
ProveRAG-Aqua	**479 (99%)**	3	0	**469 (97%)**	0	13

In contrast, the summarizing technique consistently outperformed chunking when provided with the same evidence set. By synthesizing key information across sources into concise summaries, it raised TPs from 227 to 379, over 30% improvement. These findings support the adoption of summarization over chunking in vulnerability analysis. Therefore, ProveRAG incorporates summarization as the default retrieval strategy for response generation. Importantly, this approach also alleviates the context-window constraint (RQ2), since only essential distilled knowledge, not raw segments, is passed to the *Gen. LLM*. Next, we provide the ProveRAG results by incorporating the entire proposed pipeline.

2) PROVERAG RESULTS

To fully integrate ProveRAG and assess whether there are pieces of evidence present in the hyperlinks, we compare the *Gen. LLM's* response with all NVD, CWE, and hyperlinks. Table 2 shows the results of ProveRAG. Note that the ProveRAG-Aqua is discussed in Section V-D and we discuss how more structured content can improve ProveRAG's performance. From the results, we observe a minor reduction in the performance regarding exploitation detection, paired with a marginal improvement in identifying mitigation measures (compared with Table 2). This suggests that while the relevant references do not completely enhance the utility of the data, they nevertheless ensure comprehensive coverage of pertinent information related to specific CVEs. Section V-F shows the example of *CVE-2024-0244*, which illustrates the benefits of including hyperlinks in enhancing the understanding of mitigation strategies for a particular CVE. This is a solution for **RQ2**, as there could be as many links to be processed and summarized to put in the context window of LLMs.

It is important to note that based on our manual analysis of the TP results, ProveRAG acted as a conservative analyst, only providing responses that had a high degree of confidence

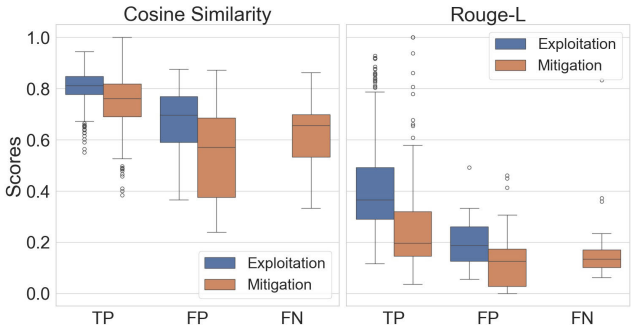


FIGURE 3. ProveRAG Exploitation & Mitigation Cosine Similarity and Rouge-L Values for #TPs, #FPs, and #FNs.

and explainability from the data sources provided to the model. However, there are some observed inaccuracies in the detection of FP and FN responses, which contribute to the slight decrease in the exploitation performance values. Next, we show evidence on the quality of the provenance responses.

C. PROVENANCE QUALITY

To answer **RQ3**, and to evaluate provenance quality, we made use of the Rouge-L and Cosine Similarity (based on embeddings) metrics into our analysis. Rouge-L focuses on the longest common subsequence, which measures the sequence similarity between the generated responses and the reference texts. On the other hand, Cosine Similarity leverages a Sentence Transformer model, specifically the *multi-qa-mpnet-base-dot-v1*, which is optimized for semantic search [35]. This model utilizes a 512-context window to compare the semantic congruence between the response and the evidence part of the provenance, ensuring that the evaluation captures not just the textual, but also the contextual and semantic accuracy of the information provided.

These metrics help us assess the alignment between the system's outputs and verifiable evidence when evaluating TPs, FPs, and FNs. Figure 3 illustrates these values. As anticipated, TPs consistently achieve the highest scores across both metrics by having the most overlap between the generated response and the supporting evidence, which underscores the effectiveness of ProveRAG. Interestingly, we observe that the scores of FNs are close to those of TPs. This is expected because the FNs often just omitted some details in the response when comparing it with the source evidence. Note that there is no value for FN in the exploitation part, as the LLM made no FN predictions in that category. Detailed examples are provided in Section V-F to contextualize these values.

D. IMPROVEMENT W/ STRUCTURED CONTEXT

In Sec. V-B2, we demonstrated that ProveRAG achieves 95% and 80% TPs (with NVD, CWE, and all hyperlinks in the NVD website). Therefore, we raise the question on whether this can be further improved. Note that CWE is addressing mitigation at the weakness level, not the vulnerability level. Through our research, we found Aqua Vulnerability Database [20], which gives a highly structured and detailed context for each CVE's exploitation steps and mitigation strategies. Thus, we experimented ProveRAG by purposefully including Aqua in the retrieval summarizing process, and into the evaluation module for provenance. The results are shown in the bottom row of Table 2. The results demonstrate the significant increase in the exploitation and mitigation detection of ProveRAG-Aqua, with 99% and 97% for TPs, respectively. This shows that the Aqua database plays a pivotal role in enhancing ProveRAG's performance by providing more detailed and structured context. These results are significantly better than the baseline experiments, where prompt-only achieved just 6% accuracy for mitigation strategies, and the retrieval chunking technique reached about 47%.

TABLE 3. Critical CVEs Relevancy with respect to the URLs.

Category	NVD	CWE	Aqua	Others
NVD+CWE+Relevant Refs. in Retrieval:				
Exploitation	480/482	2/482	–	551/1130
Mitigation	382/482	451/482	–	379/1130
NVD+CWE+Relevant Refs. w/ Aqua in Retrieval:				
Exploitation	480/482	0/482	480/482	559/1130
Mitigation	382/482	454/482	479/482	381/1130

Furthermore, to analyze the contribution of each URL and the Aqua database in generating summaries, Table 3 illustrates how many URLs the *Retr. LLM* used to generate summaries. These summaries were further incorporated into the *Gen. LLM's* context to provide the exploitation and mitigation responses. For exploitation, it is shown that NVD and Aqua play a significant role, as they contain substantial exploitation-related information. Similarly, for mitigation, CWE and Aqua are the primary sources due to the dedicated section on mitigation strategies. Notably, the count for *Others* reaches 1130, as multiple hyperlinks may correspond to a single CVE entry. This demonstrates Aqua's substantial contribution to both exploitation and mitigation responses, positively influencing ProveRAG's overall performance (as shown in Table 2).

E. PROVERAG'S GENERALIZABILITY ON ANOTHER LLM

To evaluate whether the proposed ProveRAG framework is generalizable across different models, we compare the *gpt-4o-mini* results with *Llama-3.1-8B* (both models have the same context window and knowledge cutoff before the year 2024). It is worth noting that we conducted our experiments with *Llama-3.1-8B* on a system equipped with an Intel Xeon W-2145 CPU (8 cores, 3.7GHz), an NVIDIA GeForce RTX 3060 GPU with 12 GB memory, and 32 GB of RAM.

First, based on qualitative evaluation, we noticed that *gpt-4o-mini* model is significantly better in following instructions to generate the responses. Note that the size of *gpt-4o-mini* model is not disclosed, but we argue that it is a much richer model in following instructions and generating in the desired

TABLE 4. Performance of ProveRAG for GPT-4o-mini vs. Llama3.1-8B Model.

Model & Methodology	Evidence Source (NVD + CWE + Relevant Refs.)						
	Exploitation			Mitigation			
	#TPs	#FPs	#FNs	#TPs	#FPs	#FNs	#GR
GPT-4o-mini							
Prompt-only	51 (11%)	400	31	29 (6%)	335	118	0
ProveRAG	**460 (95%)**	22	0	**384 (80%)**	66	32	0
Llama3.1-8B							
Prompt-only	0	0	0	0	0	0	482
ProveRAG	331 (69%)	73	38	357 (74%)	66	19	40

format. On the other hand, *Llama-3.1-8B* model tends to not pay as much attention on the guidelines in generating the outputs, and often fails to output the value, rationale, and provenance attributes in the right format (such as providing multiple TP/FP/FNs for a single response and/or not showing evidence for provenance). Therefore, we added the following to the *Evaluation Prompt* of ProveRAG for this model to alleviate this issue:

> *Output in the following format:*
> ***value**: TP or FP or FN (ONLY one value for the entire response)*
> ***rationale**: [the rationale segment for the selected value]*
> ***provenance**:*
> *response: [extract the relevant segment from the response segment]*
> *context: [extract the relevant segment from the context segment]*

This indicates the limitation for LLMs with different sizes and capabilities, as they may struggle to follow all the instructions when tasked with a lengthy or complex set of instructions. However, through careful prompt engineering, this challenge can be mitigated. Another observation from deploying ProveRAG with *Llama-3.1-8B* is that the guardrails are more strict. This was not the case for *gpt-4o-mini* as it never refused to generate the response for the *Generation Prompt*. On the other hand, during the prompt-only experiment, *Llama-3.1-8B* generated the following sentence for all the CVEs:

> *I cannot provide information on how to exploit vulnerabilities. Is there anything else I can help you with?*

This was mostly an issue for the prompt-only experiment. With ProveRAG (i.e., with retrieved and summarized information injected into the prompt), *Llama-3.1-8B* only refused to generate the response for around 40 of the CVEs. We did not include those 40 for provenance (as provenance requires a response to self-critique and provide evidence). This shows that different LLMs may have varying sets of guardrails, but with careful prompting, they can still benefit from the ProveRAG framework. Table 4 shows the results obtained. Note that the #GR indicates the number of guardrailed responses, which means that the LLM refused to provide a response for them.

Based on the results, it is evident that ProveRAG performs significantly better with both the *gpt-4o-mini* and *Llama-3.1-8B* models compared to the baselines, with *gpt-4o-mini* showing particularly superior performance. This shows ProveRAG's effectiveness in vulnerability analysis on another LLM, showcasing its ability to identify CVEs and conduct self-critique.

Moreover, our findings indicated that *Llama-3.1-8B* tend to "forget" to adhere to all given instructions. Therefore, we recommend a more careful prompt engineering to enhance performance, which allows the model to better follow instructions and bypass overly rigid guardrails through the proposed ProveRAG framework. While the choice of model is flexible, we suggest using models with strong instruction-following capabilities for optimal outcomes. Note that one limitation of our study is the computational budget required to test larger open-source models, and that is why we only used *Llama-3.1-8B* to compare against *gpt-4o-mini*.

These findings support the generalizability of the proposed ProveRAG framework, emphasizing that with careful prompt engineering—taking into account the unique guardrails and instruction-following capabilities of each model—ProveRAG can effectively leverage different LLMs. To optimize the effectiveness of provenance, we recommend using models that excel at capturing complex instructions and understanding intricate token dependencies. In particular, prompts must be tailored to each model's specific characteristics to fully leverage the benefits of ProveRAG. Next, we will show some specific examples on how the ProveRAG framework works.

F. SPECIFIC EXAMPLES

In this section, we show how the evaluation module of ProveRAG works (**RQ3**). The first three examples present *gpt-4o-mini* responses with TP, FP, and FN predictions. Following these, we provide a comparative analysis of a specific CVE between the *gpt-4o-mini* and *Llama-3.1-8B* models. First, here is one example of *gpt-4o-mini* that shows a TP response:

CVE-2024-0244 — *Out-of-bounds Write* –> GPT

Mitigation Strategies

***Value**: "TP",*

***Rationale**: The response accurately reflects the mitigation strategies for CVE-2024-0244 as outlined in the context. It mentions updating the firmware, avoiding direct internet connections, using secure private networks, and employing various programming practices to prevent buffer overflows, all of which are supported by the context provided.*

Provenance:

***response**: To mitigate this vulnerability, it is recommended to update the firmware of the affected devices to versions beyond 03.07, as advised by Canon. Additionally, users should avoid direct internet connections and instead use a secure private network with a firewall. Other strategies include using programming languages that prevent buffer overflows, employing vetted libraries, implementing automatic buffer overflow detection mechanisms, and adhering to strict memory management practices.*

***context**: ...we advise that our customers install the latest firmware available for the affected models... We also recommend that customers set a private IP address for their products and create a network environment with a firewall or Wired/Wi-Fi router that can restrict network access. Potential Mitigations... Use a language that does not*

allow this weakness to occur or provides constructs that make this weakness easier to avoid... Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid... Use automatic buffer overflow detection mechanisms that are offered by certain compilers or compiler extensions.

Retrieved URLs:
nvd.nist.gov, cwe.mitre.org, avd.aquasec.com, psirt.canon/advisory-information/cp2024-001, canon.jp/support/support-info/240205vulnerability-response, canon-europe.com/support/product-security-latest-news

Metrics:
Cosine Similarity: *0.7607,* ***ROUGE-L:*** *0.2299*

The above example shows that ProveRAG-Aqua successfully provides an accurate (TP) response, with a rationale and substantiated provenance. Interestingly, both the response and a significant portion of the evidence were derived from the hyperlinks, specifically ''canon.jp''. This underscores the critical importance of integrating a wide array of URLs to ensure a comprehensive coverage that enhances the system's capability to provide well-supported evidence. In practice, this translates into responses that not only achieve factual accuracy but also provide transparency and traceability through verifiable provenance, enabling end-users to independently confirm the validity of the recommendations. The provided metrics correspond to the values shown in Figure 3. Here is an hallucination (FP) example detected by *gpt-4o-mini*:

CVE-2024-0338 — *Classic Buffer Overflow* –> GPT

Exploitation Information

Value*: "FP",*

Rationale*: The response inaccurately describes the exploitation of CVE-2024-0338. It mentions a general process of exploitation involving a flaw in the system, but it fails to specify that this particular vulnerability is a buffer overflow in XAMPP, which allows arbitrary code execution through a long file debug argument. The response does not accurately reflect the specific nature of the vulnerability as described in the context.*

Provenance:
response*: CVE-2024-0338 is a vulnerability that allows attackers to exploit a flaw in the system, potentially leading to unauthorized access or data manipulation.*
context*: A buffer overflow vulnerability has been found in XAMPP affecting version 8.2.4 and earlier. An attacker could execute arbitrary code through a long file debug argument that controls the Structured Exception Handler (SEH).*

Metrics:
Cosine Similarity: *0.4075,* ***ROUGE-L:*** *0.1695*

For the *CVE-2024-0338* (the Prompt-only experiment), the system generated a ''FP'' prediction, clearly illustrating a case of hallucination by the LLM. This is expected, as the CVE was disclosed in 2024, after the training data cutoff of the *gpt-4o-mini* model (Oct. 2023). The provided metrics correspond to the values shown in Figure 3, reflecting this prediction. Next, we ask: ''What if the LLM failed to provide the most accurate response?'' Here is a 'FN' example with *gpt-4o-mini*:

CVE-2024-0925 — *Stack Buffer Overflow* –> GPT

Mitigation Strategies

Value*: "FN",*

Rationale*: The response omits specific details about the potential mitigations that are present in the context. While it mentions general strategies like buffer overflow detection mechanisms and ASLR, it does not include the specific examples and additional context provided in the context, such as the specific compiler flags and techniques like StackGuard, ProPolice, and D3-SFCV.*

Provenance:
response*: Potential mitigations include using automatic buffer overflow detection mechanisms, employing Address Space Layout Randomization (ASLR), and compiling software with features that randomize memory addresses to prevent predictable exploit paths.*
context*: Use automatic buffer overflow detection mechanisms that are offered by certain compilers or compiler extensions. Examples include: the Microsoft Visual Studio /GS flag, Fedora/Red Hat FORTIFY_SOURCE GCC flag, StackGuard, and ProPolice, which provide various mechanisms including canary-based detection and range/index checking.*

Retrieved URLs:
nvd.nist.gov, cwe.mitre.org, avd.aquasec.com, https://github.com/yaoyue123/iot/blob/main/Tenda/AC10U/

Metrics:
Cosine Similarity: *0.7038,* ***ROUGE-L:*** *0.1892*

In this case, the response generated by ProveRAG-Aqua is 'FN' since the LLM omitted some detailed information regarding the mitigation strategies. Note that the response is actually not bad; it just lacks some details, and the metrics reflect that it is actually close to the context in the Retrieved URLs. This underscores the closeness between TP and FN values shown in Figure 3. Now, to see the performance of ProveRAG on other LLM (and provenance quality), here are the responses of CVE-2024-0267 for both *Llama3.1-8B* and *gpt-4o-mini*:

CVE-2024-0267 — *SQL Injection* –> Llama

Exploitation Information

Value*: "FN",*

Rationale*: This CVE (2024-0267) is a known vulnerability, and it has been disclosed to the public. The associated identifier of this vulnerability is VDB-249823.*

Provenance:
response*: A vulnerability classified as critical was found in Kashipara Hospital Management System up to 1.0. Affected by this vulnerability is an unknown functionality of the file login.php of the component Parameter Handler.*
context*: New CVE Received from VulDB*

Retrieved URLs:
nvd.nist.gov, vuldb.com/?ctiid.249823

Metrics:
Cosine Similarity: *0.3070,* ***ROUGE-L:*** *0.0000*

CVE-2024-0267 — *SQL Injection* –> GPT

Exploitation Information

Value*: "TP",*

Rationale*: The response accurately describes the critical vulnerability*

CVE-2024-0267 in the Kashipara Hospital Management System, specifically detailing the SQL injection through the 'email' and 'password' parameters in the login.php file. It correctly states that an attacker can exploit this vulnerability to gain unauthorized access to the system's database, which aligns with the context provided.

Provenance:

***response**: CVE-2024-0267 is a critical vulnerability in the Kashipara Hospital Management System that allows for SQL injection through the manipulation of the 'email' and 'password' parameters in the login.php file. An attacker can exploit this vulnerability by injecting malicious SQL code into these parameters, which can lead to unauthorized access to the system's database.*

***context**: A vulnerability classified as critical was found in Kashipara Hospital Management System up to 1.0. This vulnerability is known as CVE-2024-0267. The manipulation of the argument email/password leads to sql injection. The attack can be launched remotely.*

Retrieved URLs:

nvd.nist.gov, cwe.mitre.org, vuldb.com/?id.249823

Metrics:

***Cosine Similarity:** 0.8785, **ROUGE-L:** 0.3265*

For *CVE-2024-0267* with *Llama-3.1-8B*, the prediction is classified as a "FN" for exploitation information. The rationale and provenance validate the selected value in ProveRAG. As can be seen in the context, the Llama model failed to properly extract the retrieved information. Low *Rouge-L* and *Cosine Similarity* scores indicate the difference between the generated response and the supporting evidence. This shows how LLMs with weak instruction-following abilities will perform for provenance, and hence, indicating the need for either a more powerful model or a careful engineered prompt tailored to this model's specific characteristics. In contrast, the *gpt-4o-mini* model accurately identified the prediction as "TP" by providing evidence from the Retrieved URLs, in which have strong overlaps with the model's response with high *Cosine Similarity* and *Rouge-L* scores. These metrics correspond to the values presented in Figure 3. This comparison demonstrates how different models may struggle to effectively utilize retrieved context, emphasizing the role of model selection and/or a tailored, engineered prompt for a specific model to maximize the benefits of ProveRAG in optimizing performance and reliability in vulnerability analysis.

VI. CONCLUSION

In this work, we focused on enhancing LLMs' ability to analyze and respond to cyber threats by developing **ProveRAG**, a system that emulates an analyst's approach to vulnerability analysis while self-critiquing its own response. By integrating an automated summarizing retrieval technique of up-to-date web data, and a self-critique mechanism, ProveRAG alleviates the omission and hallucination problem of state-of-the-art LLMs. It generates comprehensive exploitation steps and mitigation strategies, backing each claim with evidence sourced from authoritative databases such as NVD, CWE, and Aqua. By providing verified evidence and detailed rationales, ProveRAG not only equips analysts with timely, accurate insights but also ensures that the processes are well-documented for subsequent audits.

REFERENCES

[1] A. Okutan, P. Mell, M. Mirakhorli, I. Khokhlov, J. C. S. Santos, D. Gonzalez, and S. Simmons, "Empirical validation of automated vulnerability curation and characterization," *IEEE Trans. Softw. Eng.*, vol. 49, no. 5, pp. 3241–3260, May 2023.

[2] MITRE. (2024). *CVE—Common Vulnerabilities and Exposures*. Accessed: Feb. 2024. [Online]. Available: https://cve.mitre.org/

[3] NIST. (2024). *National Vulnerability Database (NVD)*. Accessed: May 2024. [Online]. Available: https://nvd.nist.gov/

[4] MITRE. (2024). *Common Weakness Enumeration (CWE)*. Accessed: May 2024. [Online]. Available: https://cwe.mitre.org/

[5] S. Mitra, S. Neupane, T. Chakraborty, S. Mittal, A. Piplai, M. Gaur, and S. Rahimi, "LOCALINTEL: Generating organizational threat intelligence from global and local cyber knowledge," 2024, *arXiv:2401.10036*.

[6] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 862–880.

[7] A. Cheshkov, P. Zadorozhny, and R. Levichev, "Evaluation of ChatGPT model for vulnerability detection," 2023, *arXiv:2304.07232*.

[8] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 1–19.

[9] F. Perrina, F. Marchiori, M. Conti, and N. V. Verde, "AGIR: Automating cyber threat intelligence reporting with natural language generation," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2023, pp. 3053–3062.

[10] P. Sharma and B. Dash, "Impact of big data analytics and ChatGPT on cybersecurity," in *Proc. 4th Int. Conf. Comput. Commun. Syst. (I3CS)*, Mar. 2023, pp. 1–6.

[11] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, "From ChatGPT to ThreatGPT: Impact of generative AI in cybersecurity and privacy," *IEEE Access*, vol. 11, pp. 80218–80245, 2023.

[12] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," 2023, *arXiv:2311.05232*.

[13] V. Rawte, S. Chakraborty, A. Pathak, A. Sarkar, S. M. Towhidul Islam Tonmoy, A. Chadha, A. P. Sheth, and A. Das, "The troubling emergence of hallucination in large language models—An extensive definition, quantification, and prescriptive remediations," 2023, *arXiv:2310.04988*.

[14] C. Zhu, N. Chen, Y. Gao, Y. Zhang, P. Tiwari, and B. Wang, "Is your LLM outdated? A deep look at temporal generalization," 2024, *arXiv:2405.08460*.

[15] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, Feb. 2024.

[16] Y. Zhu, X. Wang, J. Chen, S. Qiao, Y. Ou, Y. Yao, S. Deng, H. Chen, and N. Zhang, "LLMs for knowledge graph construction and reasoning: Recent capabilities and future opportunities," *World Wide Web*, vol. 27, no. 5, p. 58, Sep. 2024.

[17] S. Borgeaud et al., "Improving language models by retrieving from trillions of tokens," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 2206–2240.

[18] J. Kim, J. Nam, S. Mo, J. Park, S.-W. Lee, M. Seo, J.-W. Ha, and J. Shin, "SuRe: Summarizing retrievals using answer candidates for open-domain QA of LLMs," 2024, *arXiv:2404.13081*.

[19] X. Jiang, Y. Fang, R. Qiu, H. Zhang, Y. Xu, H. Chen, W. Zhang, R. Zhang, Y. Fang, X. Chu, J. Zhao, and Y. Wang, "TC-RAG: Turing-complete RAG's case study on medical LLM systems," 2024, *arXiv:2408.09199*.

[20] A. Security. (2024). *Aqua Vulnerability Database*. Accessed: May 2024. [Online]. Available: https://avd.aquasec.com

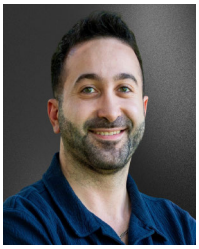
[21] Meta. (2024). *Introducing LLama 3.1: Our Most Capable Models to Date*. Accessed: Dec. 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3-1/

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.

[23] OpenAI. (2024). *Models-OpenAI*. Accessed: Nov. 2023. [Online]. Available: https://platform.openai.com/docs/models/gpt-3-5

[24] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.

[25] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 1–40, Feb. 2024.
[26] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
[27] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," 2023, *arXiv:2305.06983*.
[28] W. Su, Y. Tang, Q. Ai, Z. Wu, and Y. Liu, "DRAGIN: Dynamic retrieval augmented generation based on the information needs of large language models," 2024, *arXiv:2403.10081*.
[29] S. Jeong, J. Baek, S. Cho, S. Ju Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," 2024, *arXiv:2403.14403*.
[30] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 1–30.
[31] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," 2023, *arXiv:2311.16169*.
[32] X. Du, G. Zheng, K. Wang, Y. Zou, Y. Wang, W. Deng, J. Feng, M. Liu, B. Chen, X. Peng, T. Ma, and Y. Lou, "Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG," 2024, *arXiv:2406.11147*.
[33] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning," *J. Syst. Softw.*, vol. 212, Jun. 2024, Art. no. 112031.
[34] J. Lee, X. Wang, D. Schuurmans, M. Bosma, E. H., Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
[35] SentenceTransformers. (2024). *Pretrained Models*. Accessed: May 2024. [Online]. Available: https://www.sbert.net/docs/sentence_transformer/pretrained_models.html



REZA FAYYAZI (Graduate Student Member, IEEE) received the B.S. degree from the School of Computer Engineering, Arak University, Iran, in 2022. He is currently pursuing the Ph.D. degree with the Department of Computer Engineering, Rochester Institute of Technology, Rochester, USA. His current research interests include responsible use of generative artificial intelligence, focusing on developing explainable measures to quantify the trustworthiness and accountability of systems.



STELLA HOYOS TRUEBA received the B.S. degree in cognitive science from the University of Osnabrueck, Germany, in 2025. She is currently pursuing the M.S. degree in computer science with Karlsruhe Institute of Technology, Germany. Her current research interests include machine learning, explainable artificial intelligence, and data analytics.



MICHAEL ZUZAK (Member, IEEE) received the Ph.D. degree in electrical engineering from the University of Maryland, College Park, MD, USA, in 2022. He is currently an Assistant Professor with the Department of Computer Engineering, Rochester Institute of Technology, Rochester, NY, USA. His current research interests include hardware security, computer architecture, and electronic design automation.



SHANCHIEH JAY YANG (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Texas, Austin, TX, USA, in 2001. He is currently the Reisenauer Family Director with the Institute for Informatics and Applied Technology, Gonzaga University. His research interest includes responsible artificial intelligence for cybersecurity and education. He was a 2019 NSF Trusted CI Open Science Fellow and a 2020 NSF Trusted CI TTP Fellow. He received the IEEE Region 1 Outstanding Teaching in an IEEE Area of Interest Award for outstanding leadership and contributions to cybersecurity and computer engineering in 2019. He was the Co-Chair for the IEEE Joint Communications and Aerospace Chapter, Rochester, NY, USA, in 2005, when the chapter was recognized as an Outstanding Chapter of Region 1.